IE 643: Deep Learning - Theory and Practice **July-Dec 2026**

Project Report: Making Large Models Draw Topological Graphs

Team Name: Pensieve *Team Members: Adarsh A Rao (24B2132)*

Abstract

[**WRITE:** Abstract: the task, the approach, and the headline numbers.]

1 Introduction

[**WRITE:** Introduction: motivation, why it is hard, why LLMs, the research questions, and the report's structure (Sections 2–9).]

2 Project Workflow

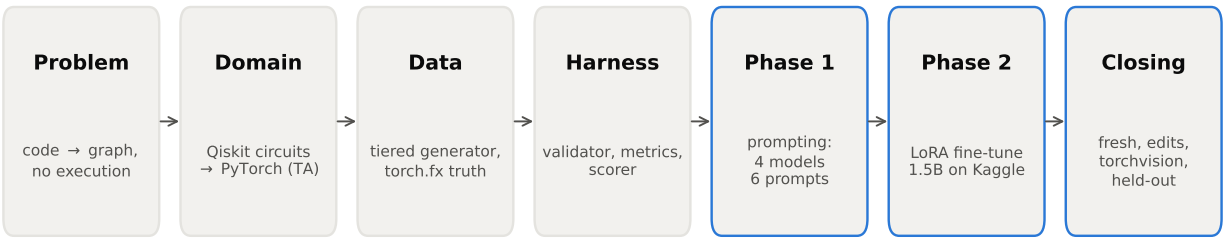


Figure 1: Project workflow, from the problem statement to the closing experiments.

[**WRITE:** Workflow: bullet points walking through Figure 1.]

3 Literature Survey

[**WRITE:** Literature survey: 3–5 papers in your own words, and how this project differs. Cite with `\citep{key}`, keys in `references.bib`.]

4 Proposed Approach or Approaches

4.1 Work done before prep-presentation review

[**WRITE:** What was built before the review, and the domain correction that followed it.]

4.2 Work done after prep-presentation review

[**WRITE:** Task formulation, the graph format, ground truth and validation, the metrics, then the two approaches: prompting (including our chain-of-simulation prompt) and LoRA fine-tuning.]

$$h = W_0 x + \frac{\alpha}{r} B A x, \qquad A \in \mathbb{R}^{r \times k},\ B \in \mathbb{R}^{d \times r},\ B_{\text{init}} = 0 \tag{1}$$

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{A}|} \sum_{t \in \mathcal{A}} \log p_\theta \left(y_t \mid y_{<t}, x \right) \tag{2}$$

4.3 Test interface

[**WRITE:** How the interface was built: inputs, the processing pipeline, outputs, and the design choices (Figure for the interface: a screenshot).]

5 Data set Details

Tier	programs	train	val	test	nodes min / mean / max	merges / graph
1 flat	400	320	40	40	7 / 10.1 / 14	0.00
2 loops	400	320	40	40	8 / 10.4 / 13	0.00
3 branches	400	320	40	40	8 / 12.0 / 16	2.02
4 blocks	400	320	40	40	12 / 15.1 / 25	1.88
all	1600	1280	160	160		

Table 1: The synthetic data set: four tiers by how much the code hides the graph. A merge is a node with more than one incoming value (residuals, concatenations).

[**WRITE:** The data: what it is, how it is generated, the tiers (Table 1), splits, preprocessing and the evaluation sets.]

6 Experiments

The team should include all experiments, even those with negative results, and address all experiments that were performed.

[**WRITE:** Experimental setup: hardware, models, decoding, the prompting runs, the training procedure and hyper-parameters (Eq. 1–2), and the experiments that failed and what was changed.]

7 Results

[**WRITE:** For each research question: what the figure/table shows, and what you infer from it, including the qualitative examples (Figures 6–8).]

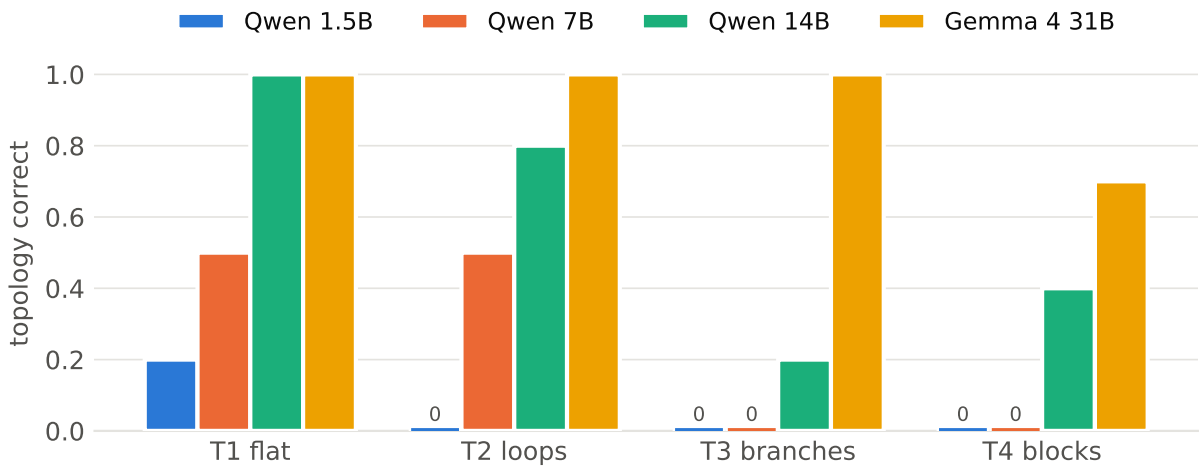


Figure 2: Zero-shot: share of the 40 test programs with the correct topology, by tier and model size.

Model (zero-shot)	T1	T2	T3	T4	topo	exact	right graph	reach F1	parsed
Qwen 1.5B	0.20	0.00	0.00	0.00	0.05	0.00	0.07	0.18	0.62
Qwen 7B	0.50	0.50	0.00	0.00	0.25	0.20	0.42	0.57	0.80
Qwen 14B	1.00	0.80	0.20	0.40	0.60	0.17	0.70	0.44	0.47
Gemma 4 31B	1.00	1.00	1.00	0.70	0.93	0.75	0.93	0.93	0.93

Table 2: Zero-shot accuracy by model size on the 40 stratified test programs (10 per tier). Tier columns and *topo* count graphs with the right topology; *exact* is strict; *right graph* also forgives spelling that does not change the computation; *reach F1* measures connectivity preservation (shared ancestor–descendant pairs).

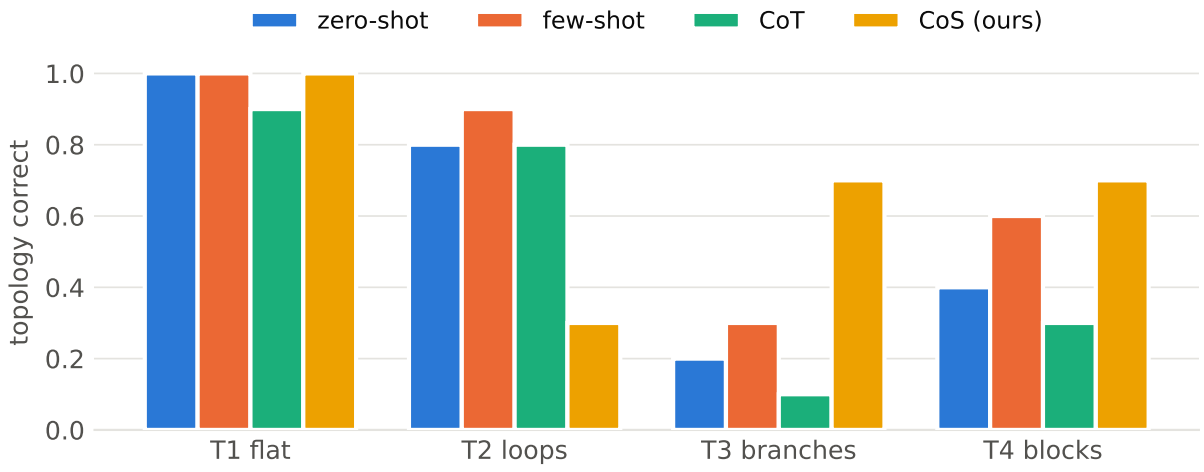


Figure 3: Prompting strategies on Qwen2.5-Coder 14B, by tier (topology correct).

Prompt (Qwen 14B)	T1	T2	T3	T4	topo	exact	right graph	parsed
zero-shot	1.00	0.80	0.20	0.40	0.60	0.17	0.70	0.47
few-shot	1.00	0.90	0.30	0.60	0.70	0.47	0.78	0.90
CoT	0.90	0.80	0.10	0.30	0.53	0.15	0.60	0.55
CoS (ours)	1.00	0.30	0.70	0.70	0.68	0.30	0.68	0.62
nodes only	1.00	0.80	0.40	0.60	0.70	0.17	–	0.47
edges only	1.00	1.00	0.40	0.60	0.75	0.75	–	1.00
full task	1.00	0.80	0.20	0.40	0.60	0.17	0.70	0.47

Table 3: Prompting strategies and the read/wire decomposition on Qwen2.5-Coder 14B, same 40 programs. *right graph* applies to whole-graph replies only.

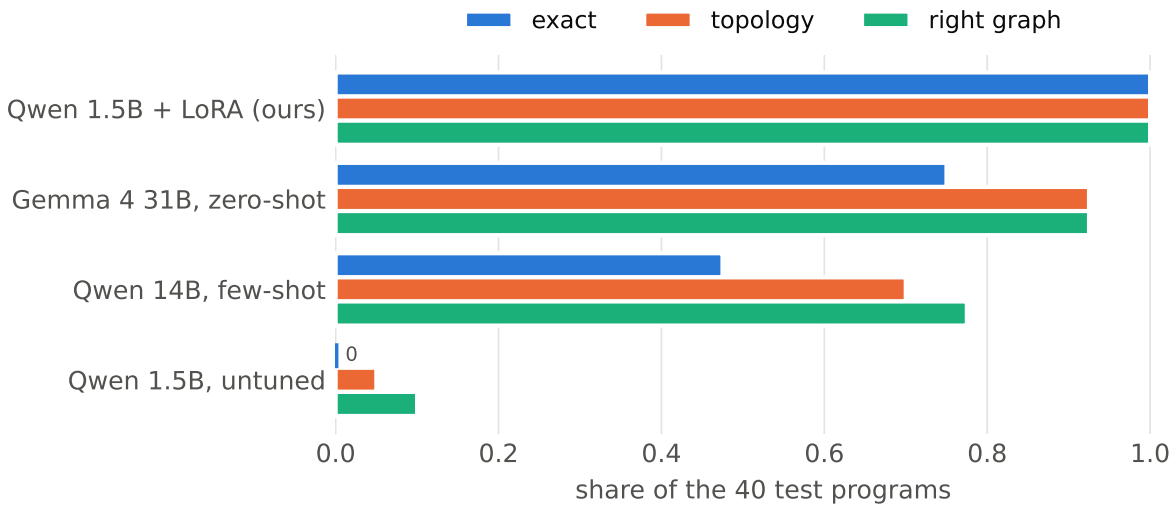


Figure 4: The LoRA-tuned 1.5B against its untuned base and the best prompted models.

Model	exact	topo	right graph	reach F1	parsed
Qwen 1.5B, untuned	0.00	0.05	0.10	0.32	0.75
Qwen 14B, few-shot	0.47	0.70	0.78	0.88	0.90
Gemma 4 31B, zero-shot	0.75	0.93	0.93	0.93	0.93
Qwen 1.5B + LoRA (ours)	1.00	1.00	1.00	1.00	1.00

Table 4: Fine-tuning against prompting, same 40 test programs and the same zero-shot prompt for the two 1.5B rows. All on our generator's programs; Table 5 has real code.

torchvision (21)	parsed	exact	topo	right	node F1	edge F1	reach F1	in-distr.	unseen	chain
Qwen 14B, prompted	14	1	1	4	0.31	0.20	0.21	4 / 5	0 / 16	2 / 14
Qwen 1.5B + LoRA	18	0	0	0	0.36	0.17	0.31	0 / 5	0 / 16	16 / 18

Table 5: RQ4, real code: counts over 21 torchvision samples. *In-distr.* (5 samples) have none of the constructs absent from all 1,280 training programs (own block classes, grouped convolutions, tuple kernels, un-shortened `F.` functions, `assert`); the rule was fixed before the tuned model's results were opened. *Right* is the right-graph level. *Chain*: readable replies with no branch at all; 4 of the 21 true graphs are chains.

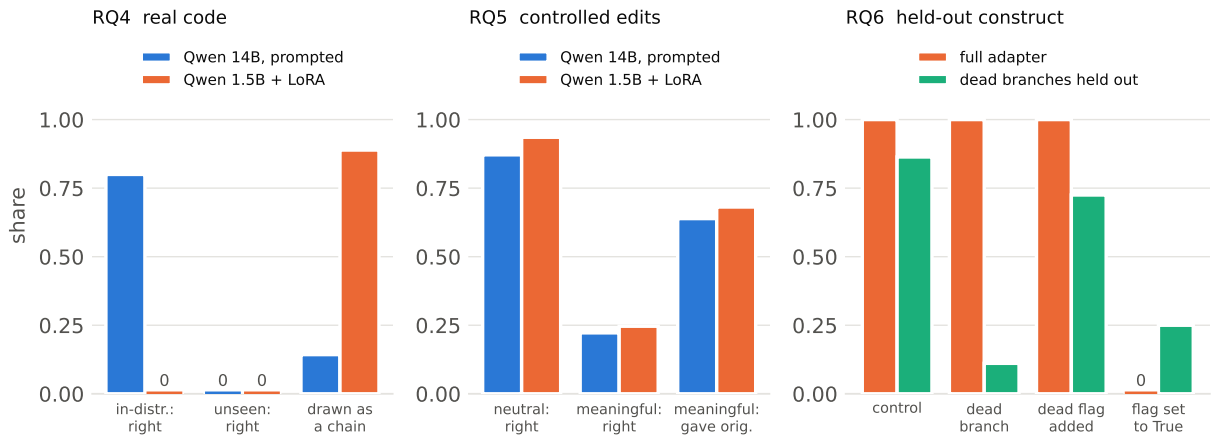


Figure 5: Closing experiments. RQ4: torchvision, right graph on in-distribution and unseen-construct samples, and the share of readable replies drawn as one straight chain. RQ5: controlled edits on programs each model solved first. RQ6: the full adapter against one trained without dead branches.

edits	solved	neutral	meaningful	gave original	reroute	swap cat	drop call
Qwen 14B, prompted	24	61 / 70	8 / 36	23 / 36	0 / 14	0 / 4	8 / 18
Qwen 1.5B + LoRA	40	116 / 124	17 / 69	47 / 69	0 / 27	0 / 9	17 / 29

Table 6: RQ5, controlled edits of the test programs, counting only the programs each model got right before the edit (*solved*). Columns count right graphs, except *gave original*: meaningful edits answered with the graph of the unedited program.

rq6 probe (84)	control	dead branch	dead flag added	flag True	dead layer in
full adapter	22 / 22	18 / 18	40 / 40	0 / 4	0 / 58
dead branches held out	19 / 22	2 / 18	29 / 40	1 / 4	18 / 58

Table 7: RQ6, a held-out construct: the full adapter against one trained without any dead-branch program (1,112 examples, one epoch against two). Group columns count topology-correct graphs. All 168 flagged branches in the training data have the flag `False`.

8 Plan for Novelty Assessment

[**WRITE:** The plan for the novelty assessment.]

9 Conclusion

[**WRITE:** Conclusion: problem, methods, results, significance, and what remains open.]

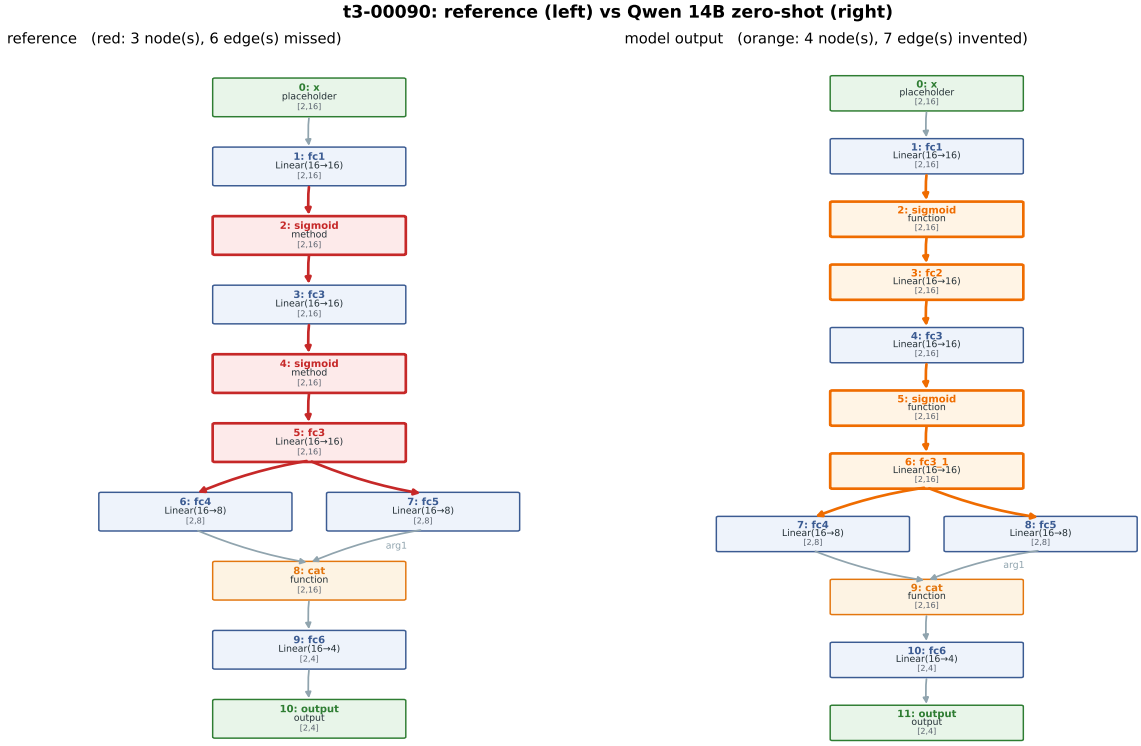


Figure 6: Qualitative: Qwen 14B zero-shot on a tier-3 test program. Red: in the reference, missed; orange: invented.

Code

The code, data generator, prompts, notebooks and results are available at [**WRITE:** GitHub link – the repository is not public yet: `https://github.com/Rao-Adarsh/graphArt_LM`].

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. `https://arxiv.org/abs/2005.14165`.

[2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. `https://arxiv.org/abs/2305.14314`.

[3] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. Talk like a graph: Encoding graphs for large language models. In *International Conference on Learning Representations (ICLR)*, 2024. `https://arxiv.org/abs/2310.04560`.

[4] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025. `https://arxiv.org/abs/2503.19786`.

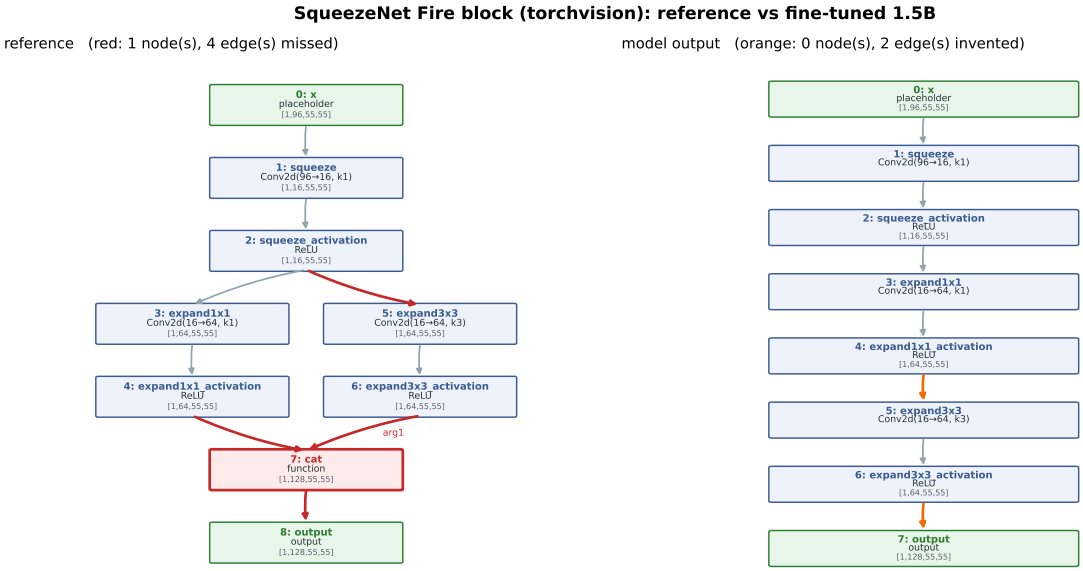


Figure 7: Qualitative: the fine-tuned 1.5B on torchvision's SqueezeNet Fire block draws a chain.

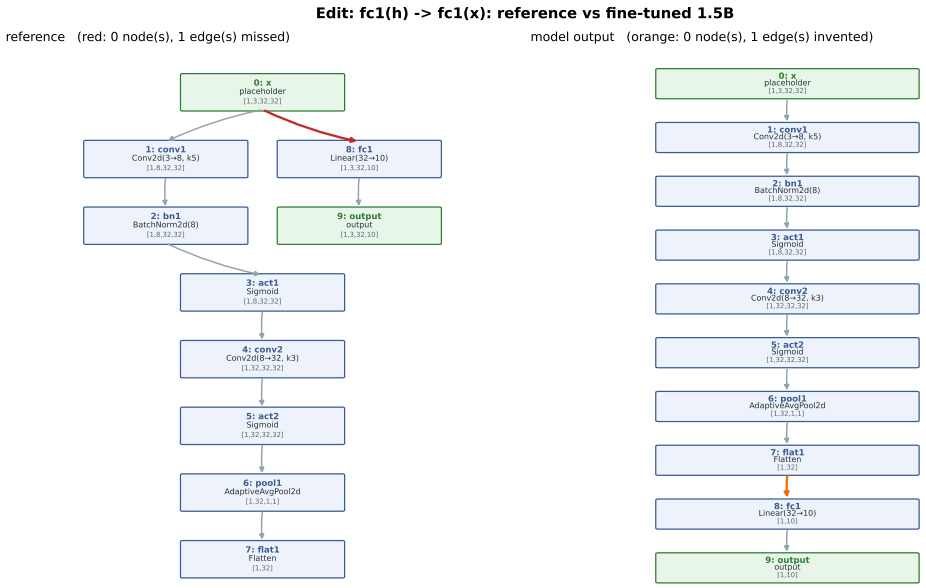


Figure 8: Qualitative: the fine-tuned 1.5B on a rerouted input answers with the unedited graph.

[5] Google DeepMind. Gemma 4 model family. `https://ollama.com/library/gemma4`. Accessed on: 26th October, 2026.

[6] Alex Gu, Baptiste Rozière, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I. Wang. CRUXEval: A benchmark for code reasoning, understanding and execution. In *International Conference on Machine Learning (ICML)*, 2024. `https://arxiv.org/abs/2401.03065`.

[7] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. Proceedings of the 7th Python in Science Conference (SciPy 2008); `https://networkx.org`, 2008. Accessed on: 26th October, 2026.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. `https://arxiv.org/abs/2106.09685`.

[9] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. `https://arxiv.org/abs/2409.12186`.

[10] Kaggle. Kaggle notebooks (free GPU sessions). `https://www.kaggle.com/code`. Accessed on: 26th October, 2026.

[11] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. `https://github.com/huggingface/peft`, 2022. Accessed on: 26th October, 2026.

[12] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021. `https://arxiv.org/abs/2112.00114`.

[13] Ollama. Ollama: run large language models locally. `https://ollama.com`. Accessed on: 26th October, 2026.

[14] OpenRouter. OpenRouter: a unified interface for LLMs. `https://openrouter.ai`. Accessed on: 26th October, 2026.

[15] James K. Reed, Zachary DeVito, Horace He, Ansley Ussery, and Jason Ansel. torch.fx: Practical program capture and transformation for deep learning in Python. In *Proceedings of Machine Learning and Systems (MLSys)*, 2022. `https://arxiv.org/abs/2112.08429`.

[16] TorchVision maintainers and contributors. TorchVision: PyTorch's computer vision library. `https://github.com/pytorch/vision`, 2016. Accessed on: 26th October, 2026.

[17] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. Can language models solve graph problems in natural language? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. `https://arxiv.org/abs/2305.10037`.

[18] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. `https://arxiv.org/abs/2201.11903`.

[19] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP 2020: System Demonstrations*, 2020. `https://arxiv.org/abs/1910.03771`.

[20] Wojciech Zaremba and Ilya Sutskever. Learning to execute. *arXiv preprint arXiv:1410.4615*, 2014. `https://arxiv.org/abs/1410.4615`.